HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY, VNU-HCM
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

Semester 261

Assignment 2 Report

Deep Learning on Large-Scale Data and Specialized Tasks

Semantic Segmentation of Underwater Imagery:
A U-Net Baseline and Pretrained DeepLabV3 on SUIM

Group Placeholder

Instructor	Le Thanh Sach
Students	**Student ID**
Tran Lam Anh	2352067
Vu Duc Viet Anh	2352074
Tran Nguyen Giap	2352284
Le Nguyen Khang	2352470

Ho Chi Minh City, 11 November 2026

Abstract

To complete: Write this last, in one paragraph of about 150 words. State the task (eight-class semantic segmentation on SUIM), the protocol (fixed split, seed, checkpoint rule, official test set), the U-Net and DeepLabV3 test mIoU and Dice, the gap between them, the outcome of the cross-entropy versus cross-entropy plus Dice experiment, the main error group, the compute trade-off, and the main limitation. Use only test-set numbers here.

Table of Contents

Abstract **i**

1 Problem and data description **1**

1.1 Problem statement 1
1.2 Problem formulation 1
1.3 Exploratory data analysis 2
1.4 Splits and leakage prevention 4
1.5 Preprocessing and augmentation 6

2 Methodology **8**

2.1 Training and evaluation pipeline 8
2.2 Model architectures 8
2.3 Fine-tuning procedure 10
2.4 Loss and evaluation metrics 11
2.5 Training configuration 12

3 Implementation results **13**

3.1 Training behavior 13
3.2 Quantitative results 14
3.3 Compute cost 15
3.4 Qualitative results 16
3.5 Controlled experiments 16
3.6 Error analysis 19
3.7 Comparison and discussion 20
3.8 Limitations and conclusion 22

AI usage disclosure **23**

References **23**

1 Problem and data description

1.1 Problem statement

Underwater robots use cameras to follow divers, avoid obstacles and inspect reefs and wrecks. These tasks require the location and extent of objects in a scene. We study semantic segmentation to assign a class to each pixel, so we can distinguish open water from obstacles and measure the area covered by reefs or sea-grass.

We use SUIM (Segmentation of Underwater IMagery), an underwater image dataset introduced by Islam et al. [3, 4]. It contains photographs from ocean exploration and human–robot experiments across several sites and water conditions. Each photograph has a manually labelled segmentation mask with eight classes: background waterbody (BW), human divers (HD), aquatic plants and sea-grass (PF), wrecks and ruins (WR), robots and instruments (RO), reefs and invertebrates (RI), fish and vertebrates (FV), and sea-floor and rocks (SR).

The dataset contains 1,635 image–mask pairs: 1,525 in the training pool and 110 in the official test set. After filtering invalid masks and training images that duplicate test images, we retain 1,540 pairs (Section 1.3). SUIM lets us compare performance across large scene regions and small objects, with robots and plants providing difficult cases because they occupy few pixels. Unlike normal images, the four challenges in underwater imagery are:

- *Colour cast.* Water absorbs red light faster than blue and green, so an object's colour changes with depth and distance. This makes reefs, rocks and sea-floor harder to distinguish by colour.
- *Turbidity.* Suspended particles scatter light and reduce contrast, which makes object boundaries harder to identify and annotate.
- *Small objects.* Divers, robots, fish and plants often occupy small regions (Section 1.3). Resizing can remove their details and thin boundaries.
- *Class imbalance.* Reefs and water cover most labelled pixels; robots account for only 0.56%. We use mIoU to give each class equal weight, since pixel accuracy can hide poor results on rare classes.

For this study, we analyse how model choice and training loss affect segmentation on SUIM, with particular attention to the small and rare classes described above. We address two questions: *First*, how does pretrained DeepLabV3 compare with U-Net trained from scratch, measured by test mIoU and per-class IoU? *Second*, does adding soft Dice to cross-entropy improve IoU for rare classes (RO, PF and WR) without reducing mIoU?

We use fixed data splits and select each checkpoint by validation mIoU. For the loss comparison, we keep the DeepLabV3 model and other training settings fixed. The U-Net comparison changes both architecture and initialization, so we cannot isolate the effect of pretraining.

1.2 Problem formulation

We represent each RGB image as $x \in \mathbb{R}^{3\times H\times W}$ and its annotated mask as $y \in \{0,\dots,7\}^{H\times W}$, where H and W are the image height and width. Each mask value identifies one of the eight SUIM classes. We resize the images and masks to 240×320 pixels for training and evaluation.

The segmentation model f_θ, with learned parameters θ, produces a score map $f_\theta(x) \in \mathbb{R}^{8\times H\times W}$.

At each position (u, v), we assign the class with the highest score to form the predicted mask $\hat{y}$:

$$\hat{y}_{uv} = \arg \max_{k \in \{0,\dots,7\}} f_\theta(x)_{k,uv}. \tag{1}$$

We train the model to match the annotated masks and evaluate its predictions using mIoU and per-class IoU, as defined in Section 2.4.

1.3 Exploratory data analysis

We analyse the 1,430 usable image–mask pairs in the training pool after filtering invalid masks and copies of test images (Section 1.4). We also compare the training-pool and test-set class distributions to identify differences that may affect evaluation.

Pixel distribution per class. We measure how often each class appears and how much of the labelled image area it covers (Table 1 and Figure 1). Reefs and invertebrates (RI) occupy 35.64% of the pixels, followed by waterbody background (BW) at 30.99%. Robots (RO) occupy only 0.56% and appear in 86 images. RI therefore has about 64 times as many pixels as RO. Fish and vertebrates (FV) appear in 958 images but occupy just 7.26% of the pixels: frequent appearances do not imply large regions.

This imbalance can hide poor rare-class predictions behind high pixel accuracy. Therefore, we use mIoU to weight all eight classes equally and test whether adding Dice to cross-entropy improves rare-class segmentation (Section 2.5).

The test set has a different class balance (see the last column in Table 1). BW increases from 30.99% of training-pool pixels to 41.81% of test pixels, while RI decreases from 35.64% to 19.47%. This shift may contribute to differences between validation and test scores.

Table 1: Per-class statistics. Images, pixel share and median area are computed on the usable train and validation pool; median area is the class fraction of an image, over images that contain the class. The last column is the pixel share on the official test set.

Code	Class	Images	Pixels (%)	Median area (%)	Test pixels (%)
BW	Background (waterbody)	1,185	30.99	33.6	41.81
HD	Human divers	359	2.34	4.9	3.69
PF	Aquatic plants and sea-grass	218	2.29	7.4	3.08
WR	Wrecks and ruins	245	6.77	43.2	7.28
RO	Robots and instruments	86	0.56	3.8	0.90
RI	Reefs and invertebrates	964	35.64	56.1	19.47
FV	Fish and vertebrates	958	7.26	7.2	6.77
SR	Sea-floor and rocks	571	14.15	29.5	17.00

Region size and image resolution. We measure each class's share of image area in images where it appears (Figure 2). RI and wrecks and ruins (WR) have median areas of 56.1% and 43.2%. RO, HD, FV and PF occupy smaller regions, with medians between 3.8% and 7.4%. Resizing can remove details from these regions, where boundary pixels make up a larger share of the object. We examine boundary and interior IoU separately to assess this effect.

The usable pairs span 11 original image sizes (Figure 3), with 84% at 640×480. We resize all images to the same 4:3 input size (Section 1.5). This stretches images with other aspect ratios, most commonly 16:9.

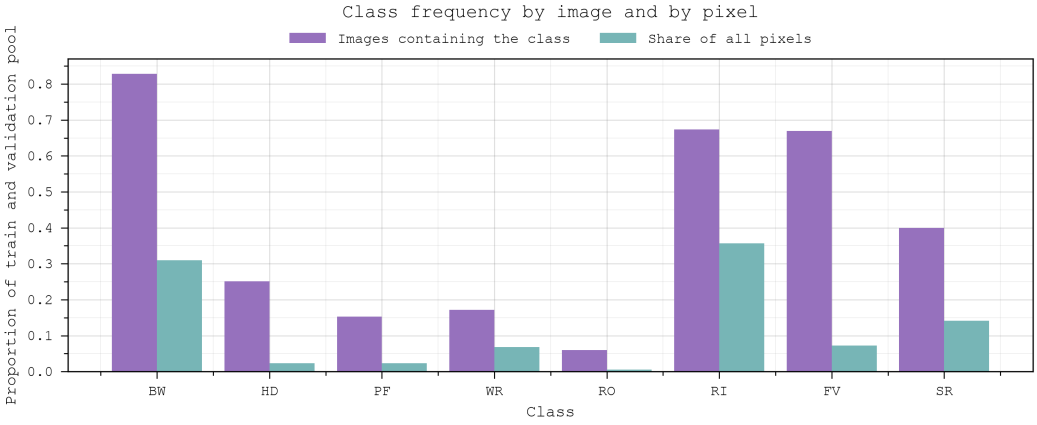


Figure 1: Pixel share and image count for each SUIM class in the usable training pool.

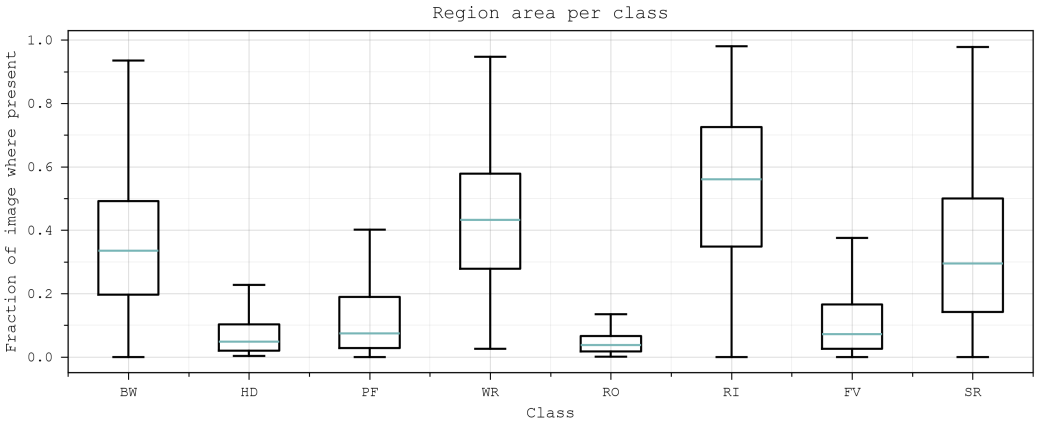


Figure 2: Class area as a fraction of the image, measured in training-pool images where the class appears.

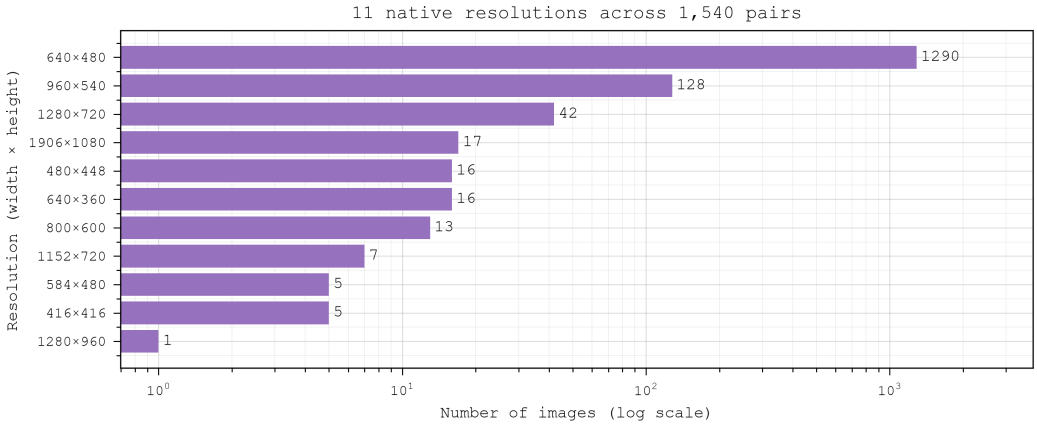


Figure 3: Original image sizes across all usable pairs, including the official test set.

Mask quality. We also audit all 1,635 pairs: 1,525 in the training pool and 110 in the test set. We match photographs and masks by filename stem and check that each file decodes. We find 0 unmatched files and 0 decoding failures. We then check every mask pixel against the eight SUIM class colours, whose RGB channels must each be 0 or 255.

We find invalid colours in 85 training-pool masks and 0 test masks and eventually exclude any pair whose mask contains an invalid colour. The rejected masks have between 2.6% and 85% invalid pixels; all remaining masks have none. A tolerance below 2.6% would therefore produce the same exclusions. All 37 image–mask size mismatches occur among these rejected pairs. For example, Figure 4 shows two rejected masks. Here, invalid pixels cover 35% and 23% of their areas, although both masks look clean at normal viewing size. These pixels form blocks along object edges, consistent with JPEG compression artefacts. The second mask also has 55 extra rows, so its labels do not align with the photograph.

We exclude these pairs because their intended labels cannot be recovered reliably. Thresholding the RGB channels would assign class codes to the damaged pixels and could introduce false rare-class labels along edges. The quality audit leaves 1,440 training-pool pairs, before duplicate filtering. It also removes genuine rare-class examples, reducing the images available to learn classes such as RO.

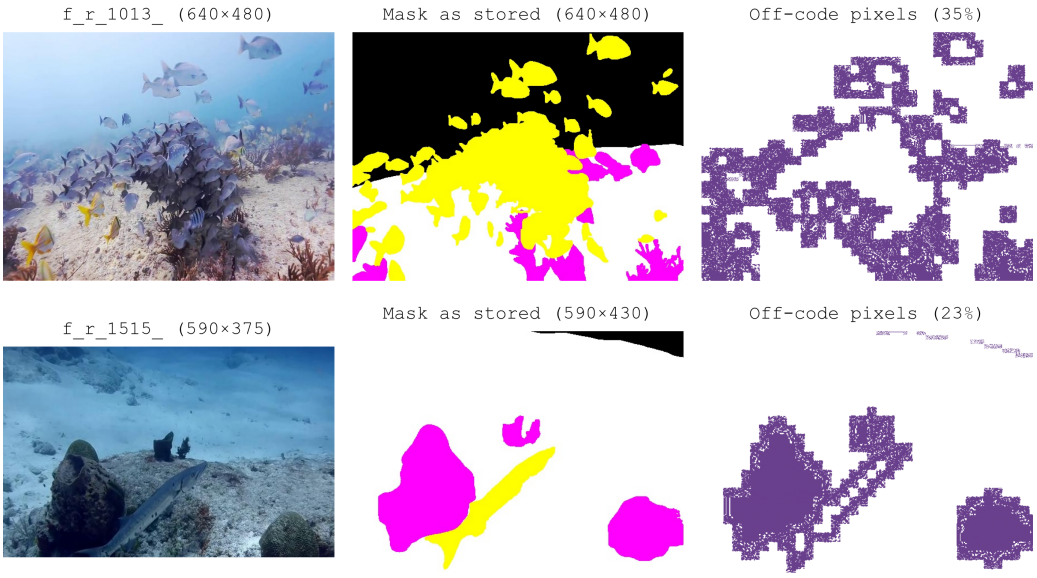


Figure 4: Two rejected masks and their invalid-colour pixels. We select the mask with the median invalid-pixel share from each group: masks matching the image size (top) and masks taller than the image (bottom).

1.4 Splits and leakage prevention

SUIM provides an official test set but no validation split. We retain all 110 test pairs and create a validation set from the training pool. We inspect the test data for quality and duplicates, but use no test labels or scores to tune hyperparameters or select checkpoints. We reserve the test set for evaluating the selected models after fixing their configurations.

Duplicates and leakage. We first check for repeated photographs within the training pool and across the training and test sets. For each image, we compute a 64-bit difference hash from

a small greyscale thumbnail. Images whose hashes differ by at most 12 bits form 2,035 candidate pairs. We then resize each candidate pair to a common size and measure its mean absolute pixel difference. We confirm a duplicate when this difference is below 5 intensity levels per channel on a 0 to 255 scale. This confirms 76 pairs, which we join into 65 groups of connected copies.

Of these groups, 10 contain images from both the training pool and the test set. For example, Figure 5 shows the pairs with the lowest and highest mean pixel differences. The first is an exact copy, with a score of 0.0. The second stores the same photograph at a different resolution and aspect ratio, yet its score of 0.5 remains below the 5 threshold. We therefore keep the test images and remove their 10 training-pool copies, leaving 1,430 pairs for training and validation.

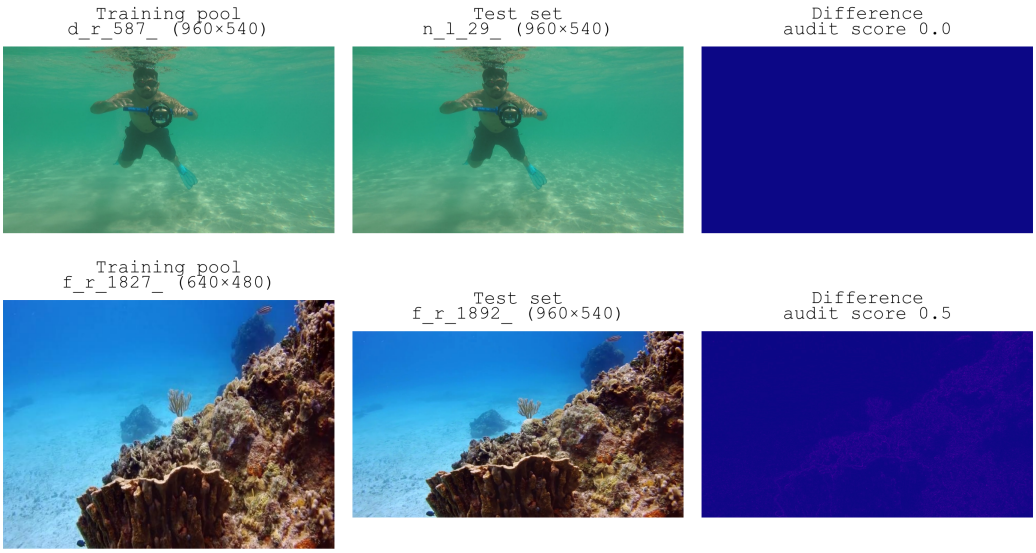


Figure 5: Training-pool copies of test images with the lowest (top) and highest (bottom) mean pixel differences among the cross-split duplicates. We resize each training image to the test image size before computing the absolute difference map. The score is the mean pixel difference on a 0 to 255 scale.

Split unit and stratification. We also find 55 duplicate groups containing 115 images entirely within the remaining training pool. We keep each group together when splitting the data, so copies of a photograph cannot appear in both training and validation. Together with the images that have no copies, these form 1,370 groups from 1,430 pairs.

To represent rare classes in both subsets, we label each image with the rarest class it contains and apply `StratifiedGroupKFold` with 5 folds, shuffling and seed 42. We use fold 0 for validation and the remaining folds for training. This gives 1,143 training images and 287 validation images, or 79.9% and 20.1% of the pool (see Table 2). All eight classes appear in both subsets, with pixel shares differing by at most 2.9 percentage points. RO, the rarest class, appears in 69 training images and 17 validation images.

We save the training and validation filenames in two files committed to the repository. Every run reads these same files, and the loader stops if a listed image is missing. This keeps split membership fixed across experiments. Both runs discussed in Section 3.1 record 287 validation images.

However, the duplicate search catches repeated photographs, but it can miss mirrored copies because the difference hash changes under a horizontal flip. It can also miss neighbouring video frames whose pixel differences exceed the threshold despite showing the same scene. In addition, SUIM includes images from EUVP, USR-248 and UFO-120, so related scenes may remain across

Table 2: SUIM splits. The official test set is held out; train and validation come from the official training pool with seed 42.

Split	Images	Share (%)	Source	Use
Train	1,143	74.2	Official train pool	Optimization
Validation	287	18.6	Official train pool	Checkpoint selection
Test	110	7.1	Official test set	Final evaluation only
Total	1,540	100.0		

splits even after we remove duplicates. Finally, only 17 validation images contain RO. Small changes in its IoU therefore provide limited evidence for choosing between models.

1.5 Preprocessing and augmentation

We load training and validation pairs from the saved split files, which keep rejected masks and test-image copies out of both subsets. For testing, we load all official test pairs. We decode each RGB mask into class indices using $c = 4\,[R \geq 128] + 2\,[G \geq 128] + [B \geq 128]$, giving 0 for BW (black) through 7 for SR (white). All masks that pass the audit contain exact class colours, so decoding preserves their labels.

We then resize each image and mask to 240×320 pixels (height by width), half the dominant resolution 640×480 in each direction. We use bilinear interpolation for images and nearest-neighbour interpolation for masks to preserve class indices along boundaries. Images with other aspect ratios are stretched, keeping the full labelled scene. The same resize applies to validation and test masks, so we evaluate at this resolution. As a result, small regions and thin structures may lose detail.

Next, we scale image pixels to $[0, 1]$ and normalize them with the ImageNet channel mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). We use these values for the pretrained backbone and apply the same normalization to U-Net to keep the inputs consistent.

During training, we also apply a random horizontal flip with probability one half, flipping the photograph and mask together to preserve their alignment. Validation and test images receive no random augmentation. We use no colour, crop or scale augmentation and keep the original class sampling and loss weights. Therefore, the Dice term in the controlled experiment is our only training change aimed at class imbalance.

After preprocessing, each batch contains a float32 image tensor of shape $B \times 3 \times 240 \times 320$ and an int64 mask tensor of shape $B \times 240 \times 320$, with labels from 0 to 7. Both models use batches of $B = 4$ images. We accumulate gradients over 4 batches to obtain an effective batch of 16.

For example, Figure 6 shows 4 images from the DeepLabV3 training loader. Here, normalized pixel values range from -2.12 to 2.64, and the masks contain 8 of the eight classes. We apply the horizontal flip to 1 of the 4 images. The overlays show that the masks remain aligned after resizing and flipping. They also reveal an annotation issue: the partly visible animal at the right edge of the last image is labelled as sea floor.

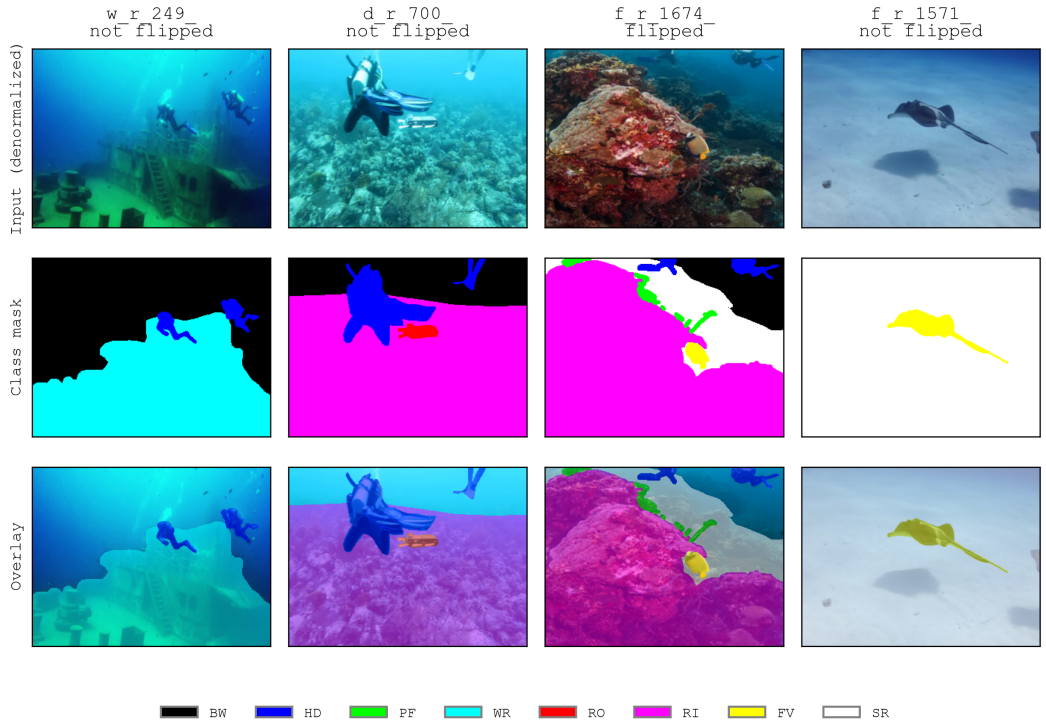


Figure 6: A preprocessed batch from the DeepLabV3 training loader. From top to bottom, the rows show the photographs with normalization undone, the SUIM class masks, and the masks overlaid on the photographs, leaving background water unblended. Each column title gives the image name and indicates whether we applied a horizontal flip.

2 Methodology

2.1 Training and evaluation pipeline

We use the same training and evaluation pipeline for both models (see Figure 7). After auditing the SUIM pairs, we preprocess the retained images and masks as described in Section 1.5 and load them in batches. Each model produces eight class scores, or logits, per pixel, which we compare with the target mask through the training loss. We accumulate gradients over several batches before updating the weights with AdamW. The learning rate increases linearly during warm-up and then follows a cosine decay (see Section 2.5 for the schedule and batch settings).

After each epoch, we evaluate the model on the validation set and save the checkpoint with the highest mIoU. To form a predicted mask, we choose the highest-scoring class at each pixel. If the score map and target mask differ in size, we first resize the scores with bilinear interpolation. Both have 240×320 pixels in these runs, so no additional resize is needed. We then pool pixel counts across the split into a confusion matrix and compute the metrics defined in Section 2.4, without further prediction refinement or test-time augmentation.

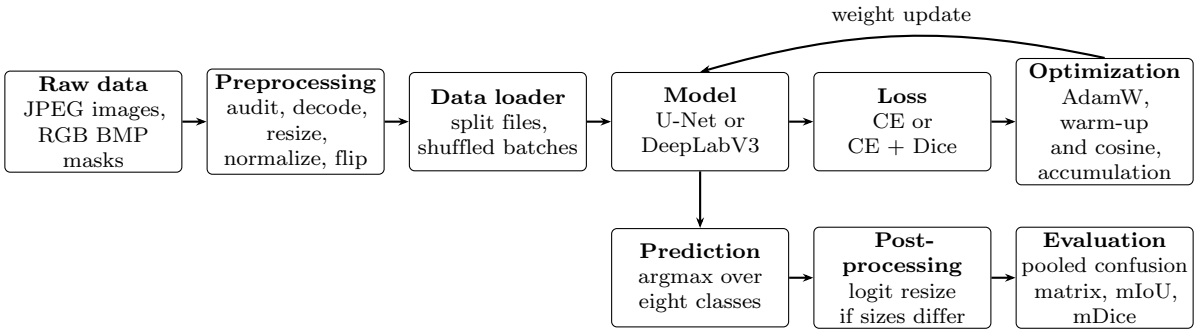


Figure 7: The shared SUIM pipeline. The training path computes the loss and updates model weights, while the evaluation path converts model scores into class masks and measures segmentation quality. We evaluate validation predictions after every epoch and reserve the official test set for the selected models.

To limit memory use, we read and preprocess images from disk as batches are requested. The 4 loader workers prepare upcoming batches while the GPU processes the current one, and pinned host memory supports transfer to the GPU. We seed shuffling with the run seed and epoch number so that batch order is reproducible, including after resuming a run. Gradient accumulation lets us keep the physical batch small while using a larger effective batch (Section 2.5). When training on multiple GPUs, we divide the training set across devices and synchronize gradients at optimizer steps, while validation uses the full split on one device.

We implement the audit and duplicate search, dataset transforms, U-Net, combined cross-entropy and Dice loss, metrics, and training loop. The loop handles gradient accumulation, learning-rate scheduling, checkpointing and resume. For the underlying operations, we use `PyTorch` for layers, automatic differentiation, optimization and data loading, while `torchvision` provides DeepLabV3 and its pretrained weights. `Pillow` handles image decoding and resizing, `NumPy` supports mask decoding, and `scikit-learn` provides the stratified group split.

2.2 Model architectures

2.2.1 U-Net baseline

The baseline is a U-Net [5] that we implemented ourselves and train from random initialization. Each encoder level applies two 3×3 convolutions, each followed by batch normalization and

a ReLU, and then halves the resolution with 2×2 max pooling. This repeats 4 times, and one more block of two convolutions forms the bottleneck. The number of channels starts at 32 and doubles at every level up to 512 at the bottleneck, where a 240×320 input has shrunk to 15×20. The decoder mirrors the encoder: a 2×2 transposed convolution with stride 2 doubles the resolution and halves the channels, a skip connection concatenates the result with the encoder features of the same resolution, and another block of two convolutions fuses them. A final 1×1 convolution maps the 32 channels to eight logits per pixel at the input resolution. All convolutions are padded, so the output has the size of the input, and both input sides must be divisible by 16, which holds for our input size. Compared with the original U-Net, our version uses padded instead of unpadded convolutions, adds batch normalization, and starts with half as many channels. It has 7,763,272 parameters. Figure 8 shows the resulting network.

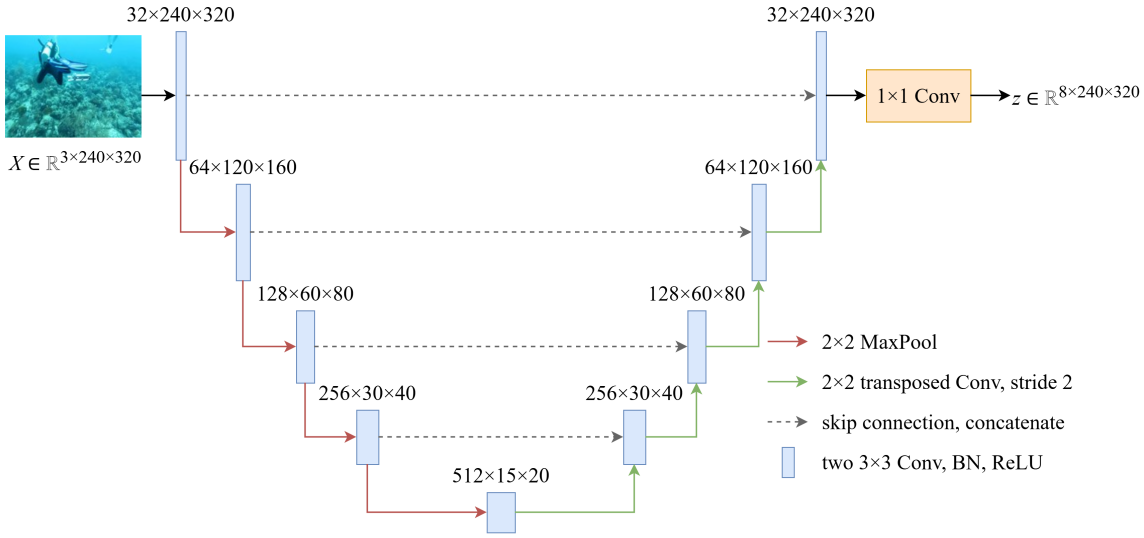


Figure 8: The U-Net baseline. Each bar is a block of two 3×3 convolutions, each followed by batch normalization and ReLU, labelled with its output channels and resolution for our input size. Every layer is written by the group and starts from random weights.

The skip connections carry full-resolution features to the output, which helps the model place boundaries and keep small objects such as fish and divers. Its main limitation for this task is that it starts without any knowledge of natural images and must learn all its features from 1,143 training images. Its context is also limited, because each bottleneck feature sees only a part of the image, so it has fewer global cues than DeepLabV3 for telling reef from rock and sea floor.

2.2.2 Pretrained DeepLabV3

We use DeepLabV3 with a ResNet-50 backbone and pretrained weights from `torchvision` [6, 7]. In the last two backbone stages, dilated convolutions replace spatial downsampling, giving an output stride of 8. An input of 240×320 pixels therefore produces a 30×40 feature map with 2,048 channels. This retains more spatial detail while allowing the filters to cover a larger region of the input. The atrous spatial pyramid pooling (ASPP) module then combines five branches: a 1×1 convolution, three 3×3 dilated convolutions with rates 12, 24 and 36, and global average pooling. Each branch produces 256 channels. We concatenate their outputs and project them back to 256 channels through a 1×1 convolution with dropout, followed by a 3×3 convolution. A final 1×1 convolution produces class scores, which we upsample bilinearly to the input size. At our feature-map height of 30 pixels, many outer filter taps at the larger dilation rates fall on padding. These branches therefore behave partly like 1×1 convolutions. The global pooling branch supplies image-level context in this setting, as discussed in the DeepLabV3 paper [6].

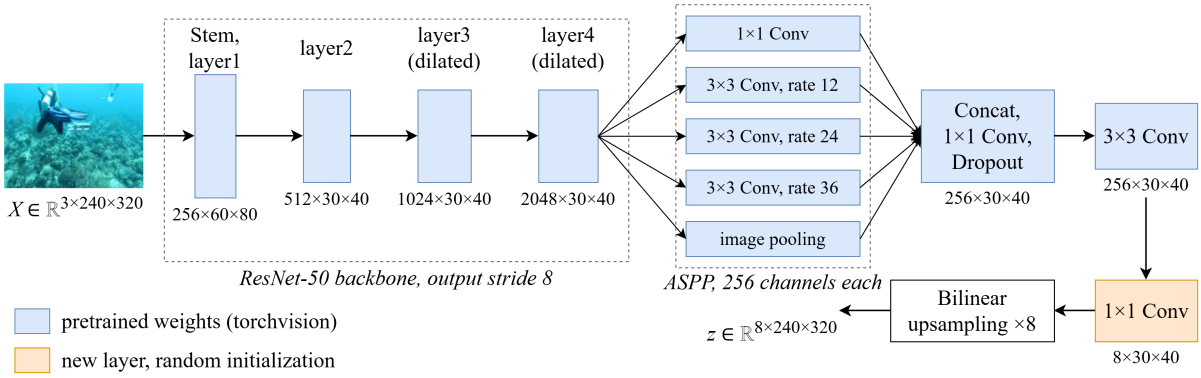


Figure 9: DeepLabV3 adapted for SUIM. Blue layers retain the pretrained initialization, while the orange output layer starts from random weights and predicts eight classes. The white box has no learned parameters. We fix batch normalization statistics throughout the network and remove the auxiliary head.

The pretrained network learns from a COCO subset containing the Pascal VOC categories, with 21 classes including background. The `torchvision` documentation reports 66.4% mIoU on COCO val2017 with VOC labels [7]. To adapt it to SUIM, we replace the final classifier with a randomly initialized eight-class 1×1 convolution and remove the auxiliary head attached to the third backbone stage. The backbone and ASPP module retain their pretrained initialization, and we fine-tune all remaining 39,635,528 parameters. The original model with both heads has 42,004,074 parameters. We implement these adaptations in a wrapper, which also fixes batch normalization statistics and assigns separate learning rates to the backbone and classifier (see Section 2.3).

Pretraining provides an initial set of image features, while ASPP combines local features with wider scene context. These features may help distinguish large regions such as reefs and sea floor. However, the model computes scores at one eighth of the input resolution and upsamples them without decoder skip connections, which can blur thin structures and object boundaries. Its pretraining data also differ from SUIM in colour cast and turbidity. We examine these limitations through class-level and boundary metrics. DeepLabV3 has 5.1 times as many parameters as our U-Net (see Table 3).

Table 3: Initialization and parameter counts of U-Net and DeepLabV3.

Model	Role	Initialization	Implementation	Parameters
U-Net	Baseline	Random	Group	7,763,272
DeepLabV3 (ResNet-50)	Pretrained	COCO, VOC labels	torchvision, new head	39,635,528

2.3 Fine-tuning procedure

We fine-tune all layers of DeepLabV3 using two learning-rate groups. The backbone uses 10^{-4}, while the head uses 10^{-3}. The head includes ASPP, the 3×3 convolution and the new output layer. We choose a lower rate for the backbone to limit changes to its pretrained features, while the higher head rate allows the classifier to adapt to SUIM's eight classes. This group includes both the pretrained ASPP layers and the randomly initialized output layer. Both rates follow the same warm-up and cosine schedule, so their ratio stays fixed throughout training (see Section 2.5). We set these rates before training and do not tune them on validation scores.

We also keep the pretrained running mean and variance fixed in every batch normalization layer,

while learning its scale and shift parameters. Each forward pass contains only 4 images, which can give noisy batch statistics. Gradient accumulation increases the effective batch for weight updates, but each normalization layer still sees the same small physical batch. Therefore, we use the stored statistics during both training and evaluation. The input images are normalized with the ImageNet channel values described in Section 1.5.

2.4 Loss and evaluation metrics

For the baseline comparison, both models use pixel-wise cross-entropy with equal class weights. The loss averages over the N pixels in each batch:

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{N}\sum_{i=1}^{N} \log p_{i,y_i}, \tag{2}$$

where $p_{i,k}$ is the softmax probability for class k at pixel i, and y_i is the target label. To examine whether an overlap-based loss helps rare classes, the controlled experiment adds soft Dice. For image n in a batch of B images and class k among the $K = 8$ classes, let $g_{i,k}$ equal one when pixel i belongs to class k and zero otherwise. Then,

$$D_{n,k} = \frac{2\sum_i p_{i,k}\, g_{i,k} + \epsilon}{\sum_i p_{i,k} + \sum_i g_{i,k} + \epsilon}, \qquad \mathcal{L}_{\mathrm{Dice}} = 1 - \frac{1}{BK}\sum_{n=1}^{B}\sum_{k=1}^{K} D_{n,k}, \tag{3}$$

where the pixel sums are taken within image n, and ϵ prevents division by zero. The combined loss weights the two terms equally: $\mathcal{L} = 0.5\,\mathcal{L}_{\mathrm{CE}} + 0.5\,\mathcal{L}_{\mathrm{Dice}}$. Because Dice weights each class equally regardless of its area, it may improve learning for classes that occupy few pixels. Computing it per image before averaging also keeps the loss independent of how batches are grouped for gradient accumulation. Classes absent from an image remain in the average. Their Dice numerator contains only ϵ, giving a score near zero unless the total predicted probability for that class is also very small. Such classes usually contribute a nearly constant loss with little gradient, so most of the Dice term's effect comes from classes present in each image.

For evaluation, predictions and target masks form a $K \times K$ confusion matrix C, where C_{jk} counts pixels of true class j predicted as class k. Counts are pooled across the entire split before calculating the scores. For each class, $\mathrm{TP}_k = C_{kk}$, $\mathrm{FP}_k = \sum_{j\neq k} C_{jk}$ and $\mathrm{FN}_k = \sum_{j\neq k} C_{kj}$, giving

$$\mathrm{IoU}_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k}, \qquad \mathrm{Dice}_k = \frac{2\,\mathrm{TP}_k}{2\,\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k}. \tag{4}$$

The reported mIoU and mean Dice average these class scores equally across all eight classes, including background. Pooling counts gives more influence to images containing larger regions of a class, while averaging per-image scores would give each image equal weight. The implementation assigns zero when a class has neither target nor predicted pixels across the split, although all eight target classes appear in our validation and test data. Evaluation uses the 240×320 mask grid described in Section 1.5. Since BW and RI dominate the pixel counts (see Section 1.3), pixel accuracy would give limited information about the rare classes, so we focus on mIoU and class-level scores.

To locate errors more precisely, we also calculate scores separately for boundaries and interiors. A target pixel belongs to the boundary if any of its four neighbours has a different class label. This marks both sides of a class transition, creating a two-pixel-wide band, while the image border alone does not count as a boundary. Separate confusion matrices for boundary and interior pixels then give IoU and Dice through the same equations. Comparing these scores reveals whether errors concentrate along object edges. Unit tests check the metric calculations against a hand-computed confusion matrix.

2.5 Training configuration

Both models train for 40 epochs with AdamW and weight decay 10^{-4} (see Table 4). The learning rate rises linearly over the first 5 epochs, then follows a cosine decay to zero, with an update after each optimizer step. U-Net uses 10^{-3} throughout the network, while DeepLabV3 uses the backbone and head rates described in Section 2.3. Each update accumulates 4 batches of 4 images, giving an effective batch of 16. Both runs use full single precision, with weight decay and random horizontal flips for regularization. DeepLabV3 also retains dropout in ASPP.

Training continues for the full schedule without early stopping. After each epoch, validation mIoU determines whether to save a new best checkpoint, which we retain for subsequent evaluation. Seed 42 controls new-layer initialization, batch order and random flips, and convolution algorithms run deterministically. The hyperparameters are fixed before training, so validation selects the checkpoint rather than tuning the configuration. This budget may limit U-Net, whose loss is still decreasing near the end of training (see Section 3.1).

U-Net runs on an RTX 4060 Laptop GPU with `PyTorch` 2.14.0 and CUDA 13.0, while DeepLabV3 runs on an RTX 4060 Ti with `PyTorch` 2.14.0 and CUDA 12.6. The table lists the settings for U-Net and the DeepLabV3 reference configuration. For the loss comparison, we keep the DeepLabV3 architecture, split, seed and training schedule fixed and change only the loss. We compare mIoU and rare-class IoU to assess whether adding Dice improves those classes without lowering the overall score. Since each configuration uses one seed, these differences describe the observed runs rather than variation across repeated training.

Table 4: Training settings of the reported runs.

Setting	U-Net	DeepLabV3
Input size	240×320	240×320
Initialization	Random	COCO, VOC labels; new head
Optimizer	AdamW	AdamW
Learning rate	10^{-3}	10^{-4} backbone, 10^{-3} head
Scheduler	5-epoch linear warm-up, cosine	5-epoch warm-up, cosine
Batch size (effective)	4 (16)	4 (16)
Epochs / seed	40 / 42	40 / 42
Weight decay	10^{-4}	10^{-4}
Augmentation	Horizontal flip	Horizontal flip
Loss	Cross-entropy	Cross-entropy
Mixed precision	No	No
Checkpoint rule	Best validation mIoU	Best validation mIoU
Hardware	RTX 4060 Laptop GPU	RTX 4060 Ti
Software	PyTorch 2.14.0, CUDA 13.0	PyTorch 2.14.0, CUDA 12.6

3 Implementation results

3.1 Training behavior

For U-Net, training loss decreases from 1.95 to 0.74 over 40 epochs, while validation loss follows a similar trend (see Figure 10). Validation loss is lower than training loss in 30 epochs and exceeds it by at most 0.02. This can occur because training loss averages predictions made as the weights change throughout an epoch, while validation uses the final weights and the stored batch normalization statistics. During warm-up, validation mIoU reaches 18.3% at epoch 5, then continues rising to 36.9% at epoch 38. This epoch also has the lowest validation loss and supplies the selected checkpoint. The final mIoU is only 0.04 percentage points below the peak.

Both losses continue decreasing near the end of training, with no widening gap that would suggest overfitting. This suggests that the fixed schedule may limit learning, although a longer run would be needed to test that explanation. At the selected checkpoint, PF and RO still have zero validation IoU. These are the classes with the fewest labelled pixels (see Table 1), so the improving overall score does not imply that all classes are learned. The run takes 14.9 minutes, averaging about 22 seconds per epoch.

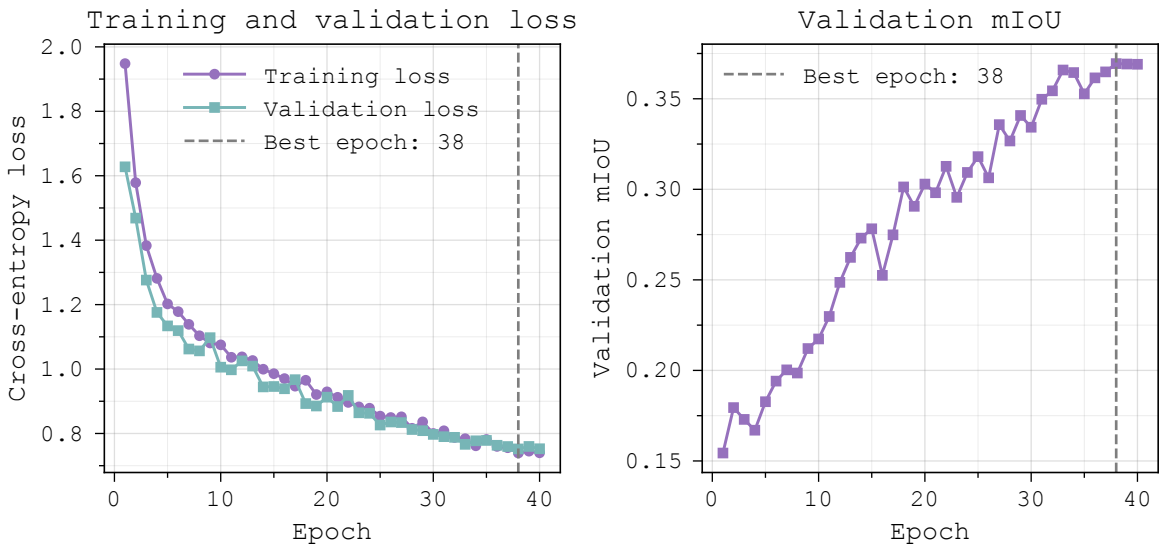


Figure 10: U-Net training and validation losses alongside validation mIoU over 40 epochs, with seed 42. Validation mIoU selects the checkpoint at epoch 38.

DeepLabV3 shows a different pattern: training loss keeps falling while validation loss begins to rise (see Figure 11). With cross-entropy, training loss decreases from 1.22 to 0.08, while validation loss reaches its minimum at epoch 11 and rises to 0.98 by epoch 40. However, validation mIoU continues improving beyond the loss minimum and selects epoch 31. The final mIoU is only 0.15 percentage points below this peak. Thus, the rising validation loss suggests overfitting in the predicted probabilities, while the overlap score remains largely stable near the end of training.

With CE + Dice, training loss falls from 1.03 to 0.13, while validation loss again reaches its minimum at epoch 11 before increasing to 1.66 (see Figure 12). Validation mIoU peaks at epoch 24 and finishes 1.32 percentage points below that peak, showing a larger late-training decline than the reference. Therefore, selecting the best validation checkpoint matters more for this run than taking the final epoch. The two losses use different objectives, so their absolute loss values should be read within each run rather than compared as a common measure. Both histories

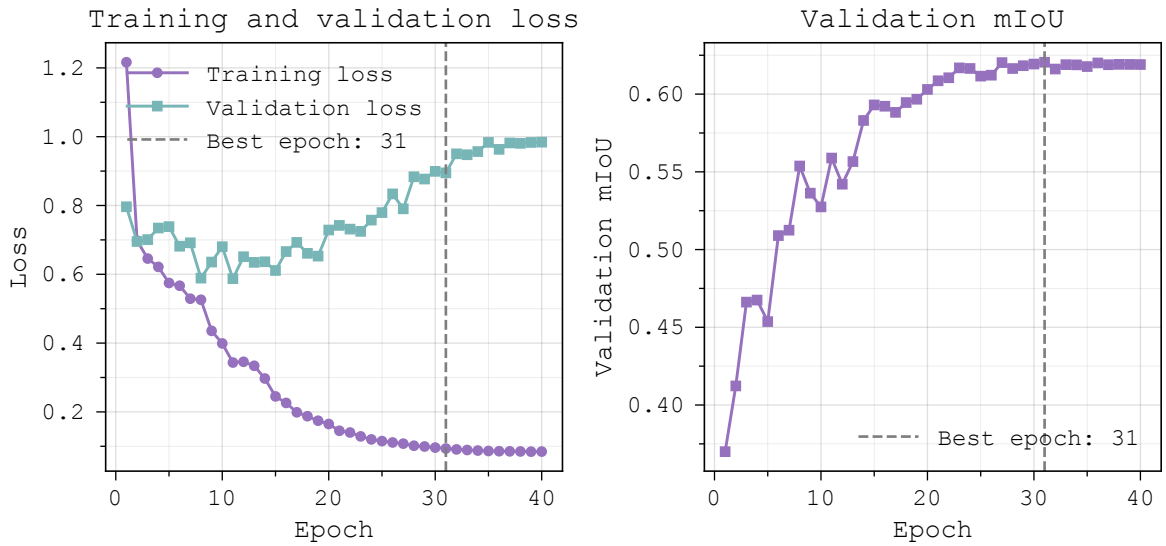


Figure 11: Training and validation losses alongside validation mIoU for the DeepLabV3 reference run with cross-entropy. The dashed line marks epoch 31, selected by validation mIoU.

cover 40 epochs, with recorded training times of 21.4 minutes for cross-entropy and 21.6 minutes for CE + Dice.

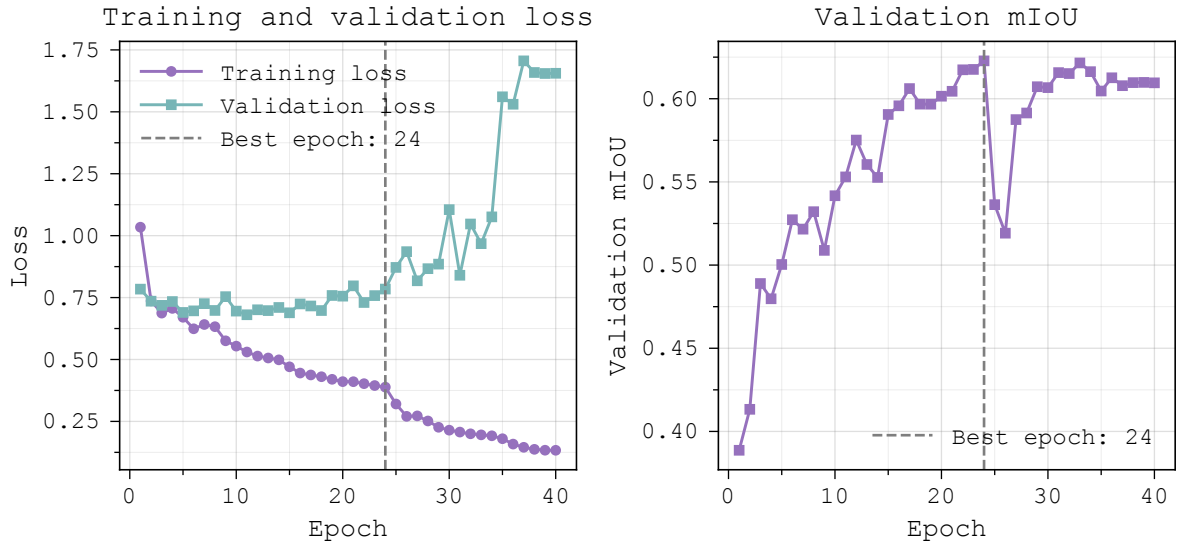


Figure 12: Training and validation losses alongside validation mIoU for DeepLabV3 with CE + Dice. Validation mIoU selects epoch 24, before the decline in the later epochs.

3.2 Quantitative results

On the same 287 validation images, DeepLabV3 with cross-entropy scores 62.40% mIoU and 74.17% mean Dice, while U-Net scores 36.94% and 48.05%, respectively (see Table 5). Each run uses seed 42 and the checkpoint with the highest validation mIoU. DeepLabV3 gains 25.46 percentage points in mIoU, but also uses 39.64 million parameters compared with U-Net's 7.76 million. Since the models differ in architecture and initialization, this comparison does not isolate

the contribution of pretraining.

Table 5: Validation results on 287 SUIM images at the selected checkpoints. Each configuration uses seed 42.

Model	Loss	mIoU (%)	Mean Dice (%)	Params (M)
U-Net	CE	36.94	48.05	7.76
DeepLabV3	CE	62.40	74.17	39.64
DeepLabV3	CE + Dice	62.27	74.12	39.64

To understand the gap, we examine the class-level scores (see Table 6). U-Net scores 80.62% IoU on water and 67.94% on reefs, yet has zero IoU for plants and robots. DeepLabV3 raises plant IoU to 12.00% and robot IoU to 71.66%, while diver IoU increases from 30.83% to 70.37% and fish IoU from 33.14% to 66.84%. These gains show that DeepLabV3 improves segmentation beyond the large water and reef regions. However, plants remain difficult, with a much lower IoU than any other class.

Table 6: Per-class validation IoU (%) on the same 287 SUIM images.

Model	BW	HD	PF	WR	RO	RI	FV	SR
U-Net (CE)	80.62	30.83	0.00	41.16	0.00	67.94	33.14	41.85
DeepLabV3 (CE)	84.19	70.37	12.00	69.08	71.66	74.47	66.84	50.59
DeepLabV3 (CE + Dice)	83.61	68.49	12.40	66.92	74.76	74.04	67.59	50.38

The loss comparison shows a smaller overall change. With CE + Dice, DeepLabV3 scores 62.27% mIoU and 74.12% mean Dice, down 0.13 and 0.05 percentage points from cross-entropy. The class scores reveal a trade-off: robot IoU rises from 71.66% to 74.76% and plant IoU from 12.00% to 12.40%, while wreck IoU drops from 69.08% to 66.92% and diver IoU from 70.37% to 68.49%. Therefore, adding Dice helps robots in this run but does not improve all three target rare classes or maintain overall mIoU. With only one seed per configuration, the small overall gap offers limited evidence for choosing between the losses. The next subsection compares their training time, memory use and inference cost.

3.3 Compute cost

DeepLabV3 uses 5.1 times as many parameters as U-Net and requires 3.4 times as many floating-point operations per image (see Table 7). Its 95.8 GFLOPs compare with U-Net's 28.3 GFLOPs at the same input size. U-Net performs much of its computation at full resolution in the first encoder and last decoder blocks, while DeepLabV3 processes most features on a smaller spatial grid. This explains why the difference in operation counts is smaller than the difference in parameter counts. We count operations with the `PyTorch` FLOP counter, treating a multiplication and an addition as two operations. These counts describe computational work, while measured latency also depends on the device and execution efficiency.

U-Net trains in 14.9 minutes on an RTX 4060 Laptop GPU, using 0.97 GiB of peak allocated GPU memory. On an RTX 4060 Ti, DeepLabV3 with cross-entropy takes 21.4 minutes and 1.93 GiB, while CE + Dice takes 21.6 minutes and 1.93 GiB. Adding Dice therefore increases training time by about 1% without changing the network's parameter count or inference operations. The runs use different GPUs, so these timings describe their observed cost rather than a hardware-matched speed comparison.

For inference, U-Net averages 4.8 ms per image, while both DeepLabV3 variants average about 7.8 ms. These measurements use batches of 4 validation images and exclude data loading. The similar latencies of the two DeepLabV3 variants are expected because the loss is used only during training. Together, the measurements show the cost accompanying the accuracy gains in Section 3.2: DeepLabV3 has a larger model and higher operation count, while adding Dice leaves inference cost essentially unchanged.

Table 7: Compute cost for 240 × 320 inputs. Latency excludes data loading and uses validation batches of 4. Memory is peak allocated GPU memory.

Model	Params (M)	GFLOPs	Train time (min)	Latency (ms/image)	Peak memory (GiB)	Training hardware
U-Net	7.76	28.3	14.9	4.8	0.97	RTX 4060 Laptop
DeepLabV3 (CE)	39.64	95.8	21.4	7.8	1.93	RTX 4060 Ti
DeepLabV3 (CE + Dice)	39.64	95.8	21.6	7.8	1.93	RTX 4060 Ti

3.4 Qualitative results

We compare reference and CE + Dice predictions on the same validation images (see Figure 13). To choose examples reproducibly, we rank images by the reference model's per-image mIoU, averaging over classes present in either the target or prediction. The figure shows the highest-scoring image, the highest-scoring image containing a robot, and the lowest-scoring image. These cases illustrate how the models behave across different scenes, while the split-wide scores in Section 3.2 describe their overall performance.

In the first image, `w_r_148_`, both predictions closely follow the large wreck region and its boundary with water. The robot image, `d_r_211_`, is more revealing: the reference recovers most of the robot body but simplifies its thin extensions, while CE + Dice assigns parts of the body to diver and fish classes. Thus, its higher aggregate robot IoU does not imply a better prediction on every robot image. In the last image, `w_r_162_`, both models label most of the scene as reef even though the annotation contains plants and wrecks. The shared failure suggests that changing the loss alone does not resolve this scene's class confusion.

3.5 Controlled experiments

We examine five choices against the DeepLabV3-ResNet50 reference, changing one factor at a time: backbone freezing, augmentation, backbone depth, loss and input resolution. All runs use the same split, seed 42, 40 epochs and an effective batch of 16. The reference fine-tunes the full network with horizontal flips, cross-entropy and 240 × 320 inputs. For the half-resolution run, we resize output scores to the common target grid before assigning labels, so all configurations are evaluated at the same mask resolution (see Table 8).

Freezing the backbone reduces mIoU by 8.18 percentage points, the largest drop among the variants, while removing horizontal flips costs 1.66 points. Fine-tuning therefore matters more than retaining a fixed pretrained feature extractor in these runs, and the simple flip augmentation also helps. Reducing the input to 120 × 160 lowers mIoU by 6.33 points and boundary mIoU from 30.42% to 25.75%. This loss is consistent with the disappearance of small structures during resizing. The class changes in Figure 14 show that freezing and reduced resolution lower IoU for every class, rather than affecting only one difficult category.

In contrast, ResNet101 gains only 0.05 mIoU points, while CE + Dice loses 0.13. The larger backbone therefore offers little overall improvement in this single-seed comparison. Dice raises

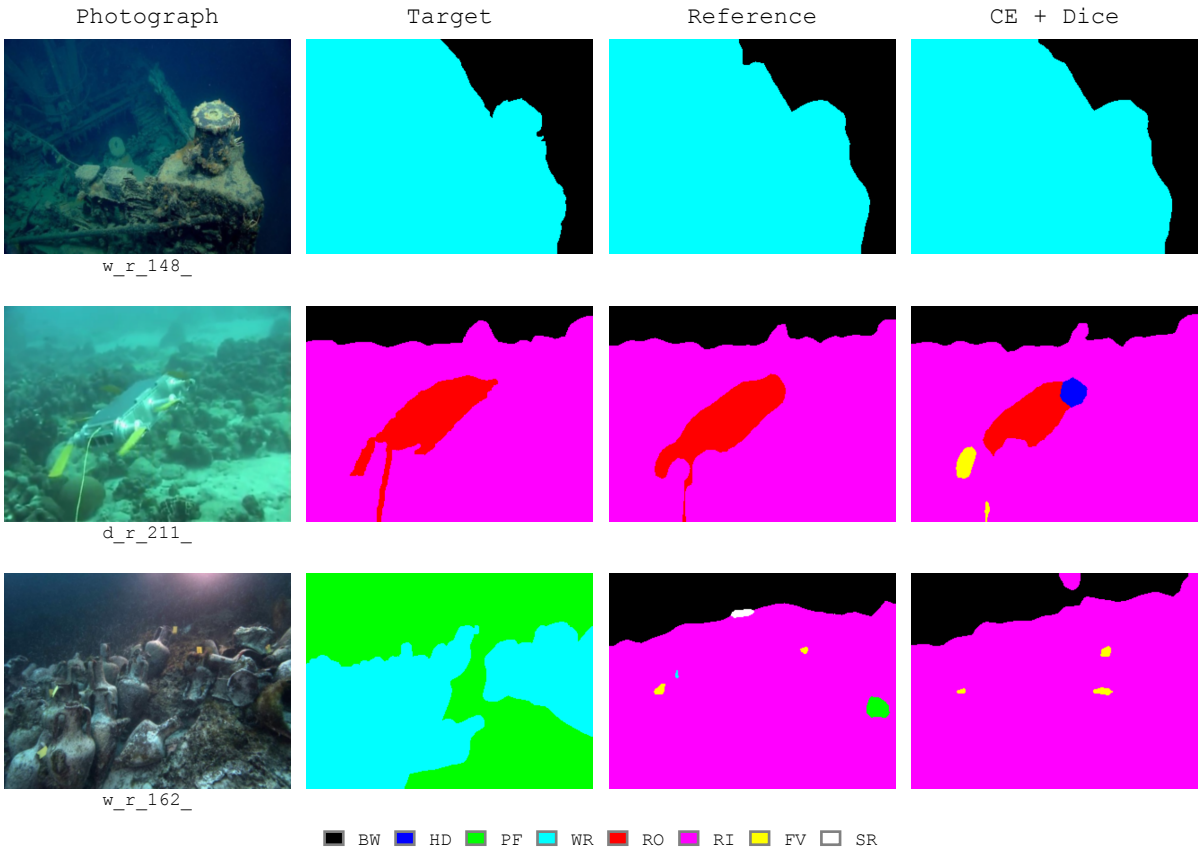


Figure 13: Matched validation predictions from the reference and CE + Dice models. Rows show the highest-scoring reference image, its highest-scoring robot-containing image, and its lowest-scoring image. Columns show the photograph, annotation and the two predictions. Colours identify SUIM classes; the legend uses the class codes introduced in Section 1.1.

Table 8: Controlled comparisons on the same validation split, with seed 42. Scores are percentages; Δ mIoU is the change from the reference in percentage points.

Configuration	mIoU	Mean Dice	Δ mIoU	Boundary mIoU
Reference	62.40	74.17	+0.00	30.42
Frozen backbone	54.22	67.43	-8.18	26.12
No augmentation	60.74	73.05	-1.66	29.48
ResNet101	62.45	73.99	+0.05	30.46
CE + Dice	62.27	74.12	-0.13	30.42
Half resolution	56.07	69.06	-6.33	25.75

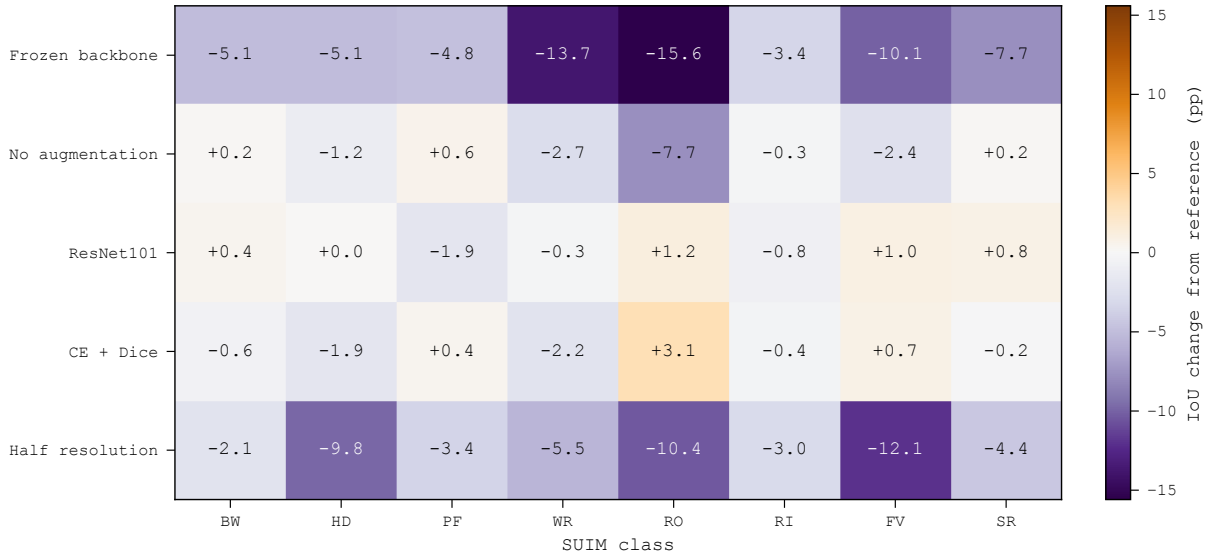


Figure 14: Per-class IoU changes from the reference, in percentage points. Positive values indicate improvement. Freezing the backbone and halving the input resolution reduce every class score, while the other variants produce smaller, mixed changes.

robot IoU by 3.10 points and plant IoU by 0.41, but lowers wreck IoU by 2.16 points, so it does not meet the goal of improving all three target rare classes without reducing mIoU. These small overall differences provide limited evidence for preferring either alternative. The recorded compute measurements also show a trade-off (see Figure 15): freezing and lower resolution reduce training time and peak memory, while ResNet101 increases both. The nearly unchanged inference cost of freezing reflects the fact that the same network still executes at prediction time. We retain the full ResNet50 reference as the simpler choice among the configurations with similar overall scores.

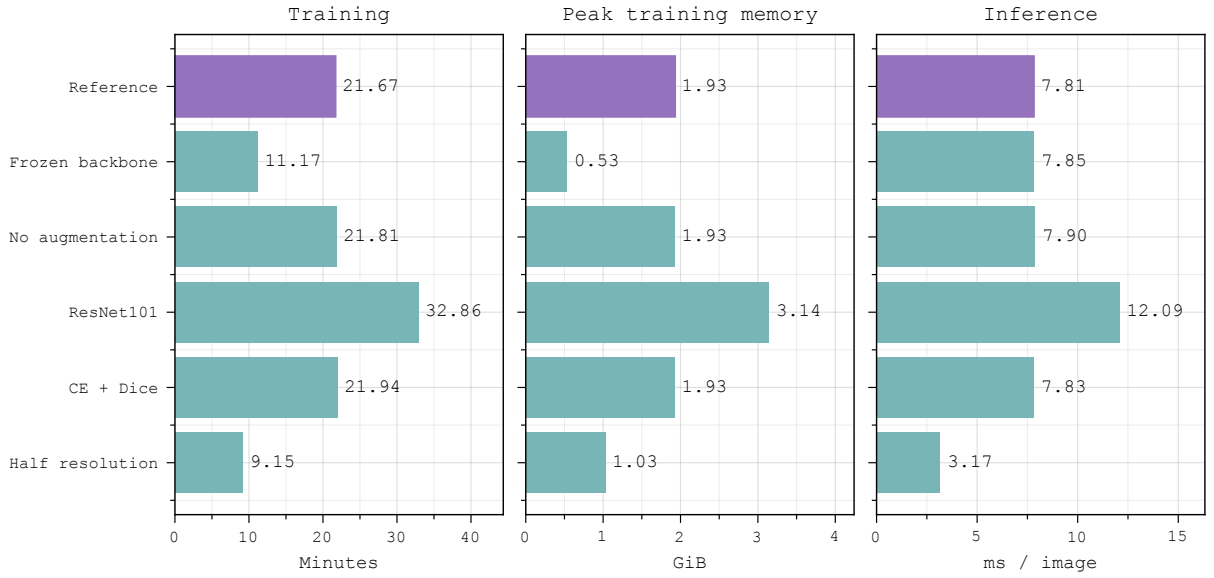


Figure 15: Training time, peak allocated GPU memory and inference latency from the saved comparison measurements. Training covers 40 epochs with validation; inference uses each configuration's input resolution. Each bar represents one run, so the chart does not estimate timing variation.

3.6 Error analysis

The reference model's largest plant confusion is with reefs: it assigns 71.27% of annotated plant pixels to RI (see Figure 16). The matrix pools all pixels from the 287 validation images, while its row normalization makes the errors of small classes visible alongside the much larger water and reef classes. Plant and reef regions can share fine, branching textures, but the matrix alone does not establish whether appearance or class imbalance causes this confusion. The plant example in Figure 17 makes the error concrete: plants cover 73.2% of the annotation, yet the model assigns no correct plant pixels and predicts mainly reef. This is a failure to distinguish classes even when the target region is large.

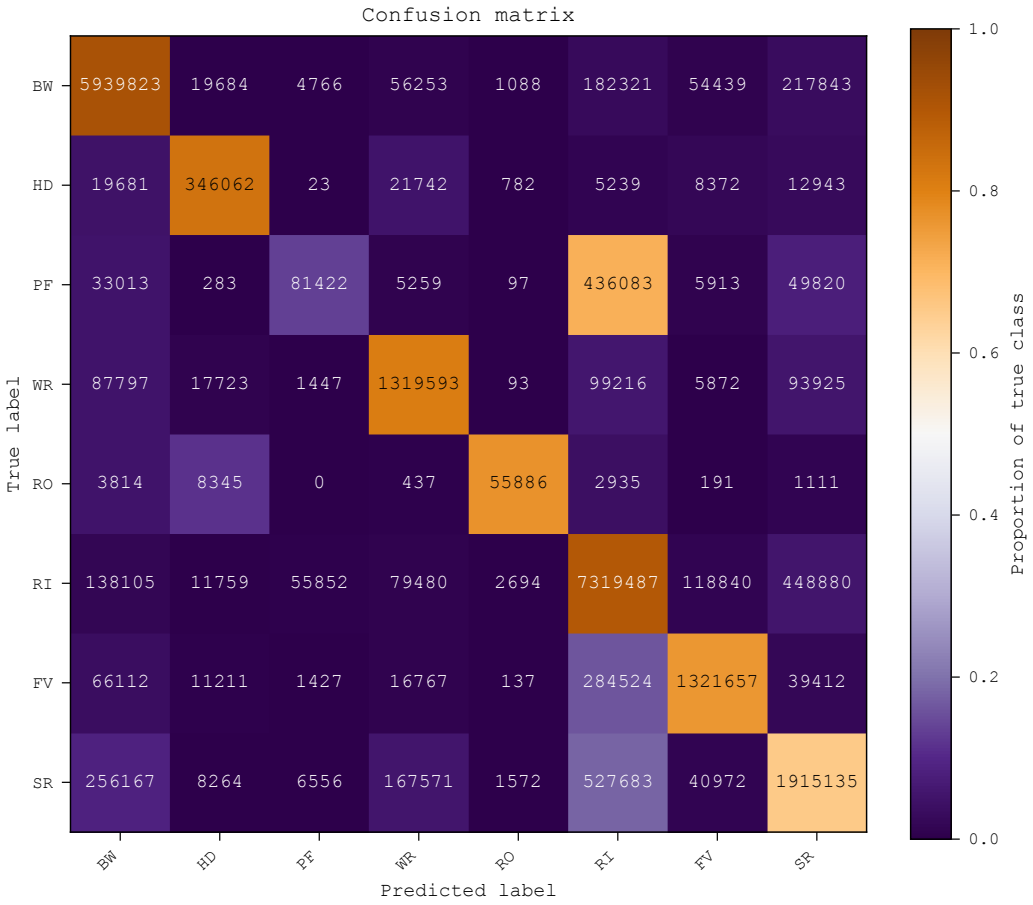


Figure 16: Reference-model confusion matrix over 287 validation images. Rows are target classes and columns are predictions. Colour shows the fraction of each target class assigned to a predicted class, while cell annotations give pixel counts.

Small objects introduce a different failure mode. Across validation images, 11.48% of robot pixels are assigned to the diver class. For example, in `d_r_798_`, the robot occupies only 3.9% of the annotation and is missed entirely, with much of its region absorbed into the nearby diver prediction (see the second row of Figure 17). The model's coarse score map and close contact between the two objects are possible contributors, although this example cannot separate their effects. We select these plant and robot failures by lowest target-class IoU, breaking ties by largest target area and then filename, rather than choosing them by appearance.

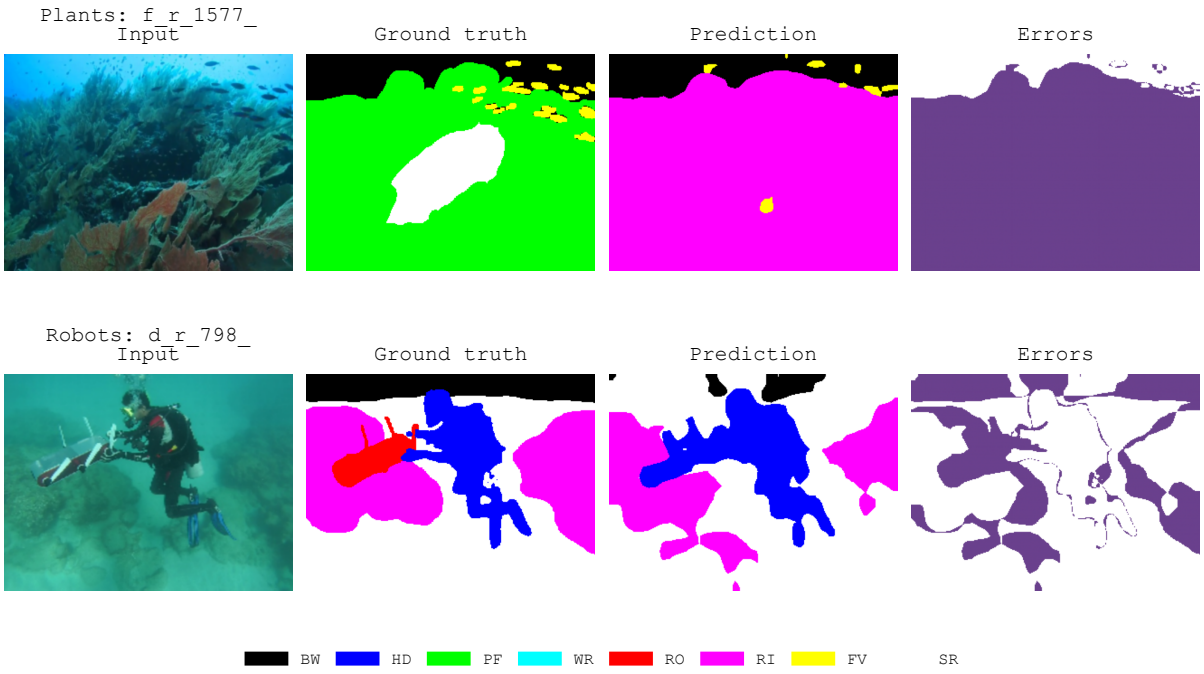


Figure 17: Reference-model failures on plants (top) and robots (bottom). Columns show the photograph, target, prediction and error map, where purple marks an incorrect pixel. The selected target class has zero IoU in each row.

Boundary scores confirm that errors also concentrate along object edges (see Figure 18). The reference reaches 30.07% mIoU at boundaries and 64.34% in interiors, while U-Net reaches 18.04% and 37.88%, respectively. Thus, both models struggle more with boundaries than with region interiors. Resizing and coarse predictions can remove thin details, while uncertain annotations can also contribute near edges. These scores quantify the gap but do not identify a single cause.

Finally, performance varies substantially across images. The reference has a median per-image mIoU of 54.43% (see Figure 19), with successful and failed cases illustrated in Section 3.4. This median gives each image equal weight and averages only over classes present in its target or prediction, so it measures a different aspect of performance from split-wide mIoU. Together, the image scores and failure panels show why one overall score is insufficient: the model can segment large, clear regions well while missing a small robot or assigning an entire plant region to reef.

3.7 Comparison and discussion

DeepLabV3 performs better than U-Net in our comparison, gaining 25.46 percentage points in validation mIoU. The gain extends beyond water and reef regions to divers, fish and robots, which U-Net segments less accurately or misses entirely. Pretrained features and wider scene context offer plausible explanations, but the comparison also changes architecture and model size. Therefore, it supports choosing this DeepLabV3 configuration for SUIM without showing how much of the improvement comes from pretraining alone. U-Net's falling losses also suggest that its fixed 40-epoch schedule may leave room for further learning.

The controlled comparisons help distinguish useful choices within DeepLabV3. Fine-tuning the backbone and retaining the original input resolution contribute more than increasing backbone depth or changing the loss. ResNet101 gives almost the same overall score at a higher compute cost, while CE + Dice trades better robot segmentation for lower scores on some other classes. The paired examples show why these aggregate gains need inspection: Dice can improve robot

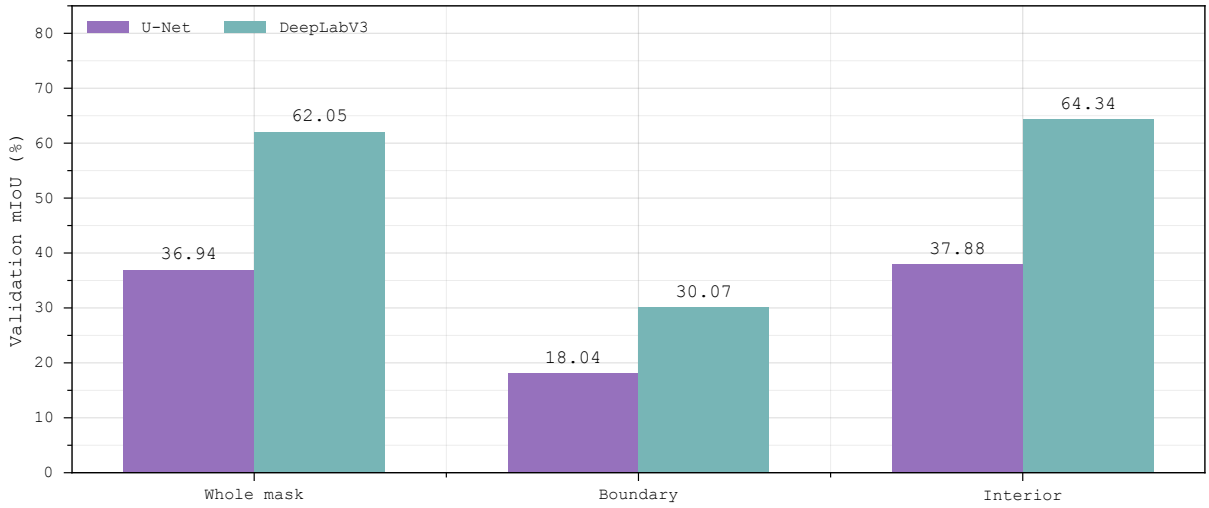


Figure 18: Whole-mask, boundary and interior validation mIoU. A boundary includes both sides of a transition between neighbouring target classes. Each region uses its own pooled confusion matrix.

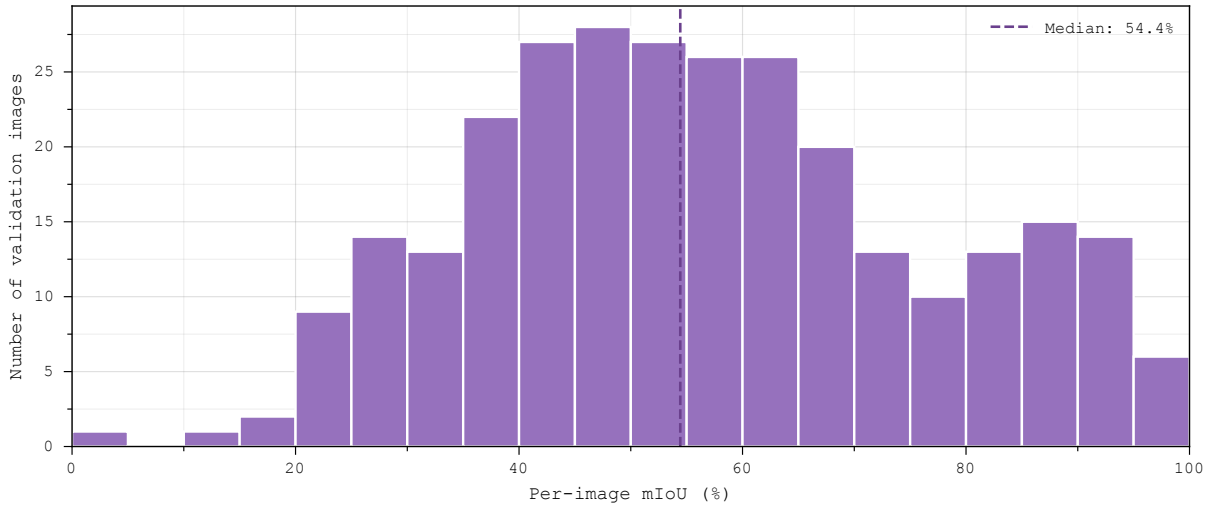


Figure 19: Distribution of reference per-image mIoU over the validation split. The dashed line marks the median. Each image score averages IoU over classes present in either the annotation or prediction.

IoU across the split yet fragment the robot in a particular image. We therefore retain ResNet50 with full fine-tuning and cross-entropy as our reference. Its simpler loss and smaller backbone avoid additional cost without a clear loss of overall accuracy among the near-tied configurations.

This choice still leaves substantial errors. Plants remain difficult, and the reference often assigns their pixels to reefs, while both models score lower at boundaries than in region interiors. These failures occur at different scales: a small robot can disappear into a neighbouring diver prediction, but a large plant region can also receive the wrong class throughout. Consequently, resolution alone cannot explain all remaining errors. DeepLabV3 also uses more parameters, operations and measured memory than U-Net, so the accuracy gain comes with a larger computational footprint. The timings describe the saved runs on different GPUs and cannot establish which architecture is faster on the same device.

3.8 Limitations and conclusion

Each configuration uses one seed, so we cannot estimate variation between training runs. This matters especially for the small differences between the reference, ResNet101 and CE + Dice, and for robot IoU, which is measured on only 17 validation images. We also use validation mIoU to select checkpoints, so the reported scores reflect that selection. The fixed training budget and untuned hyperparameters further limit the model comparison, particularly for U-Net, whose losses continue falling near the final epoch. Since architecture, initialization and parameter count change together, the results do not isolate the effect of pretraining.

The data and preprocessing introduce additional limits. Resizing masks to 240×320 pixels removes detail from thin structures, while stretching other aspect ratios changes object shapes. Duplicate filtering removes repeated photographs but can miss mirrored copies and related video frames, so related scenes may remain across splits. SUIM also covers a limited range of sites and water conditions, which restricts how far these results generalize. Rejecting invalid masks removes some genuine rare-class examples, and a single annotation per image leaves uncertainty near ambiguous boundaries. Thus, a boundary disagreement can reflect annotation uncertainty as well as a model error.

For our first research question, the pretrained DeepLabV3 configuration improves validation mIoU and every class's IoU over the U-Net baseline. For the second, adding Dice improves robot and plant IoU but lowers wreck IoU and slightly reduces overall mIoU, so it does not meet the stated goal in this run. Together with the other controlled comparisons, these findings support full fine-tuning of ResNet50 with cross-entropy at the original training resolution. The single-seed design limits the strength of that choice, especially where the overall scores are nearly equal.

AI usage disclosure

AI tools are used mainly to assist with implementation, testing, plotting and report editing. Further details are provided in `AI_USAGE.md`.

References

[1] Le Thanh Sach, *Course Project Handbook: Assignments 1–3, Deep Learning and Its Applications, CO3133, Semester 261*, HCMUT.

[2] Group Placeholder, *CO3133 Deep Learning course project repository*. https://github.com/vietanh1802/CO3133-Deep-Learning-261-Placeholder.

[3] Md Jahidul Islam, Chelsey Edge, Yuyang Xiao, Peigen Luo, Muntaqim Mehtaz, Christopher Morse, Sadman Sakib Enan, and Junaed Sattar, *Semantic Segmentation of Underwater Imagery: Dataset and Benchmark*, IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020. https://arxiv.org/abs/2004.01241.

[4] IRVLab, University of Minnesota, *SUIM dataset*. https://irvlab.cs.umn.edu/resources/suim-dataset. Accessed 02 Oct 2026.

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI, 2015. https://arxiv.org/abs/1505.04597.

[6] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam, *Rethinking Atrous Convolution for Semantic Image Segmentation*, arXiv:1706.05587, 2017. https://arxiv.org/abs/1706.05587.

[7] PyTorch contributors, *torchvision segmentation models: DeepLabV3*. https://docs.pytorch.org/vision/stable/models/deeplabv3.html. torchvision 0.29.0. Accessed 09 Nov 2026.